# Lecture 12: More Dynamic Programming

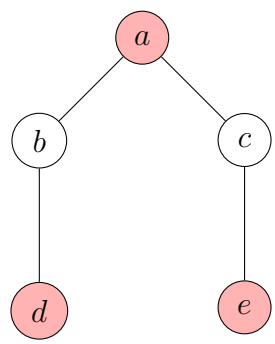
Independent Sets in Trees · Knapsack Problem

## 1 Independent Sets in Trees

### 1.1 Independent Sets: Definitions

**Definition 1.1** (Independent Set)**.** Given a graph $G = (V, E)$, an **independent set** $I$ is a subset of $V$ that induces no edges. That is:

$$I \subseteq V \text{ such that } \forall u, v \in I,\ (u, v) \notin E$$




Example: 3 vertices $\{a, c, e\}$ (highlighted) make an independent set. This is the largest independent set of this graph and so $\alpha(G) = 3$.

**Definition 1.2** (Independence Number)**.** The **independence number** $\alpha(G)$ is the size of the largest independent set in $G$.

### 1.2 Applications

Independent sets capture situations with pairwise constraints or conflicts.

**Example 1.1** (Radio Tower Placement)**.** You want to place radio towers in locations in a city, but you don't want two towers to interfere with each other.

**Model:** Represent this as a graph where:

- Vertices are possible locations
- Edges between locations that would interfere with each other

Valid placement $\Leftrightarrow$ independent set.

*Remark* 1.1. In mathematics, the **sphere packing problem** is an independent set problem.

## 1.3 Computational Problem

**Max Independent Set Problem**: Given $G$, compute $\alpha(G)$ and find an independent set of maximum size.
**Bad news:** This is a very hard problem in general (NP-hard).

**What do we do when the problem we want to solve is computationally hard?**

1. **Settle for approximate solutions** instead of optimal solutions (e.g., set cover with $O(\ln n)$ approximation)
2. **Try to get the best running time you can** even if it's not polynomial
3. **You can try to solve "typical" instances**
4. **You can solve the problem on a special class of instances**. This is what we will do today.

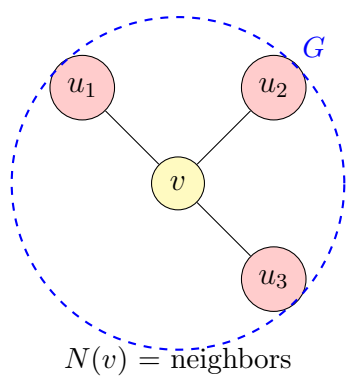
## 1.4 Solving Max Independent Set on Trees

We will solve the independent set problem on trees using dynamic programming.

### 1.4.1 Basic Idea

**Ask:** Is $v$ in a maximum independent set?

- **If yes:** $\alpha(G) = 1 + \alpha(G \setminus \{v \cup N(v)\})$
- **If no:** $\alpha(G) = \alpha(G \setminus v)$




**Notation:**

- $G \setminus v$ is $G$ with vertex $v$ removed along with all incident edges
- $G \setminus S$ is $G$ with the set of vertices $S$ removed along with all incident edges

### 1.4.2 Recurrence for General Graphs

$$\alpha(G) = \max\{1 + \alpha(G \setminus \{v \cup N(v)\}),\ \alpha(G \setminus v)\}$$

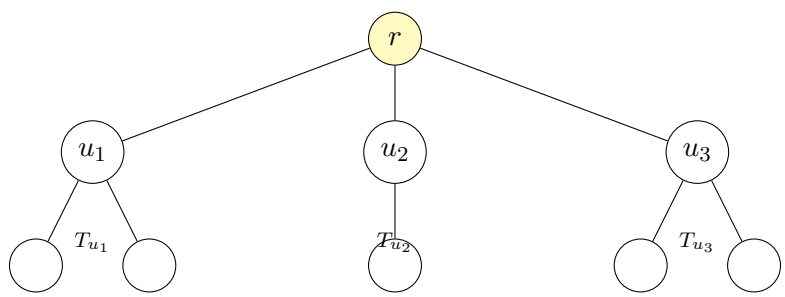
We could make a DP algorithm using this relation, but:

- Subproblems are indexed by $S \subseteq V$
- There are $2^n$ subsets — exponential!

### 1.4.3 Using Special Structure of Trees

We need to use the special structure of a tree.

**Tree structure:** $T = T_r$ where $r$ is the root. The tree splits into disjoint subtrees $T_{u_1}, T_{u_2}, \ldots, T_{u_k}$ rooted at the children of $r$.




Tree $T$ rooted at $r$ with disjoint subtrees.

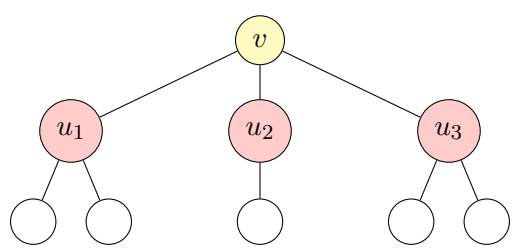
### 1.4.4 Define Subproblems

Let $\alpha(v) = \alpha(T_v)$ where $T_v$ is the subtree rooted at $v$.

**Goal:** $\alpha(T) = \alpha(T_r)$.

**How many subproblems?** $|V| = n$ (one for each vertex).

### 1.4.5 Main Identity




$$\boxed{\alpha(v) = \max\left\{1 + \sum_{w \text{ grandchild of } v} \alpha(w),\ \sum_{u \text{ child of } v} \alpha(u)\right\}}$$

**Interpretation:**

- **First case:** Include $v$ in the independent set (then must exclude all children, can include grandchildren)
- **Second case:** Don't include $v$ (can include children)

### 1.4.6 DAG of Subproblems

The dependencies form a DAG where each node depends on its children and grandchildren, so directed edges go up the tree from descendants to ancestors. We can solve the subproblems from the leaves up.

### 1.4.7 Base Cases

**Base case:** $\alpha(u) = 1$ if $u$ is a leaf.

### 1.4.8 Running Time

**Running time:** $O(n)$ — each node appears at most 3 times in computation (as itself, as a child, and as a grandchild).

## 1.5 Algorithm

```
Perform post-order traversal of tree T rooted at r
for each vertex v in post-order do
    if v is a leaf then
        α(v) ← 1
    else
        include ← 1 + Σ_{w grandchild of v} α(w)
        exclude ← Σ_{u child of v} α(u)
        α(v) ← max(include, exclude)
    end if
end for
return α(r)
```

# 2 Knapsack Problem

## 2.1 Problem Statement

**Definition 2.1** (Knapsack Problem)**. Setup:** Given $n$ items $1, 2, \ldots, n$ where:

- Each has a value $v_i$ (positive integer)
- Each has a weight $w_i$ (positive integer)

You have a capacity (budget) $W$ and you want to fill a collection $J$ of items to maximize

$$\sum_{i \in J} v_i \quad \text{subject to} \quad \sum_{i \in J} w_i \leq W$$

**Input:** List of values and weights, and the integer $W$.

## 2.2 Two Versions

1. **Version 1 (Unbounded):** We allow multiple copies of each item. $J$ is a multiset.
2. **Version 2 (0/1 Knapsack):** Only one copy of each item allowed. $J$ is a subset.

# 3 Version 1: Unbounded Knapsack

## 3.1 Dynamic Programming Approach

To construct a solution, we consider all options for including an item.

### 3.1.1 Define Subproblems

Let $V(t)$ be the maximum knapsack value with budget $t$.

**Original problem:** $V(W)$.

**How many subproblems?** $W + 1$ (for $t = 0, 1, \ldots, W$).

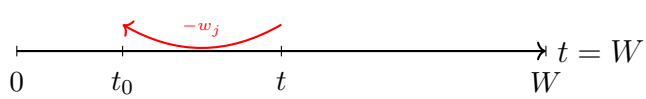
### 3.1.2 Base Case

$V(0) = 0$ (no capacity means no value).

### 3.1.3 Recurrence

$$\boxed{V(t) = \max_{j:w_j \leq t} \{v_j + V(t - w_j)\}}$$

**Interpretation:** Try adding each item $j$ (if it fits) and take the maximum value.




Try all items $j$ with $w_j \leq t$

## 3.2 Algorithm

```
for t = 0, ..., W do
    if t = 0 then
        V(t) ← 0
    else
        V(t) ← max_{j:w_j ≤ t} {v_j + V(t − w_j)}
    end if
end for
return V(W)
```

### 3.3 Running Time

**Running time:**

- Number of subproblems: $O(W)$
- Time per subproblem: $O(n)$ (try each item)
- **Total:** $O(nW)$ **time**

*Remark* 3.1. This is **polynomial in** $n$ but **exponential in the number of bits to represent** $W$.

If $W$ requires $b$ bits, then $W = 2^b$, so the algorithm runs in $O(n \cdot 2^b)$ time — exponential in input size!

This is called a **pseudo-polynomial** algorithm.

## 4 Version 2: 0/1 Knapsack

### 4.1 Why Different?

We need to keep track of more information: **which items have already been used**.

**Idea:** Go through items one by one and decide whether to use each one.

### 4.2 Define Subproblems

Let $V(i, t)$ = maximum knapsack value with budget $t$ and using items in $\{1, 2, \ldots, i\}$.

**Original problem:** $V(n, W)$.

### 4.3 Base Cases

$$\begin{aligned} V(i, 0) &= 0 \quad \text{for all } i \\ V(0, t) &= 0 \quad \text{for all } t \end{aligned}$$

### 4.4 Number of Subproblems

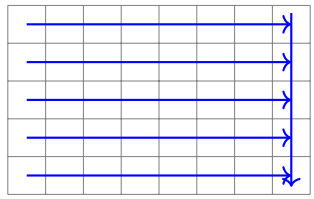
$O(nW) = (n + 1)(W + 1)$ subproblems.

### 4.5 Recurrence

$$\boxed{V(i, t) = \max \begin{cases} v_i + V(i - 1, t - w_i) & \text{if } w_i \leq t \text{ (include item } i) \\ V(i - 1, t) & \text{(don't include item } i) \end{cases}}$$

### 4.6 Fill Order

Fill the table row by row (or column by column).



Fill left to right, bottom to top

### 4.7 Algorithm

```
for i = 0, ..., n do
    for t = 0, ..., W do
        if i = 0 then
            V(i, t) ← 0
        else if t = 0 then
            V(i, t) ← 0
        else
            if w_i ≤ t then
                V(i, t) ← max{v_i + V(i − 1, t − w_i), V(i − 1, t)}
            else
                V(i, t) ← V(i − 1, t)
            end if
        end if
    end for
end for
return V(n, W)
```

### 4.8 Running Time

- Number of subproblems: $O(nW)$
- Time per subproblem: $O(1)$
- **Total:** $O(nW)$ **time**

Again, this is pseudo-polynomial.

## 5 Optimization: Sparse Subproblems

*Remark* 5.1 (Optimization for Special Cases). **Q:** What if all $w_i$'s are multiples of 10?

**Answer:** Only subproblems $V(i, t)$ where $t$ is a multiple of 10 are needed.
This reduces the number of subproblems significantly!

# 6 Memoization: Implement Dynamic Programming by Recursion

*Remark* 6.1 (Memoization). An alternative way to implement DP:

- Store solutions to subproblems in a hash table
- At each step, check if you've already solved a subproblem
- If yes, use the stored solution; if no, compute and store it

**Advantages:**

- Can be good if only a few subproblems are actually needed
- More natural to code (follows recursive structure)
- May have larger constant factors

## 6.1 Memoized Knapsack

```
Initialize hash table memo
function KnapsackMemo(i, t)
    if (i, t) ∈ memo then
        return memo[(i, t)]
    end if
    if i = 0 or t = 0 then
        return 0
    end if
    if w_i ≤ t then
        result ← max{v_i + KnapsackMemo(i − 1, t − w_i), KnapsackMemo(i − 1, t)}
    else
        result ← KnapsackMemo(i − 1, t)
    end if
    memo[(i, t)] ← result
    return result
end function
```

# 7 Summary

| **Problem** | **Subproblems** | **Time Complexity** |
|---|---|---|
| Indep. Set (Trees) | $\alpha(v)$ for each $v$ | $O(n)$ |
| Unbounded Knapsack | $V(t)$ for $0 \leq t \leq W$ | $O(nW)$ (pseudo-poly) |
| 0/1 Knapsack | $V(i,t)$ for $0 \leq i \leq n$, $0 \leq t \leq W$ | $O(nW)$ (pseudo-poly) |